

VIRTUS-CC

Capacitação em Sistemas Embarcados
FreeRTOS e CMSIS-RTOS v2

Lista de Atividades Práticas

Prioridades, Semáforos e Mutex

Plataforma: STM32 Nucleo-F767ZI

Aluno(a): _____

Objetivos da atividade

Esta lista integra as atividades práticas da **Capacitação VIRTUS-CC em Sistemas Embarcados**, com ênfase na utilização do sistema operacional **FreeRTOS** por meio da API **CMSIS-RTOS v2**, utilizando como plataforma de desenvolvimento a **STM32 Nucleo-F767ZI**.

As atividades têm como objetivo explorar experimentalmente mecanismos fundamentais de sistemas operacionais de tempo real, particularmente aqueles relacionados ao escalonamento, sincronização, concorrência e compartilhamento de recursos entre tarefas.

Ao final das atividades, o aluno deverá ser capaz de:

- compreender o funcionamento das prioridades de tarefas;
- analisar o efeito da preempção no escalonamento;
- identificar situações de *starvation*;
- utilizar semáforos para sincronização entre tarefas;
- compreender o funcionamento de semáforos binários e contadores;
- identificar condições de corrida;
- utilizar mutex para proteção de recursos compartilhados;
- compreender o efeito de diferentes tempos de espera;
- selecionar adequadamente prioridades, semáforos e mutex em aplicações embarcadas de tempo real.

Antes de começar

Considere a seguinte interpretação dos mecanismos utilizados nesta lista:

Mecanismo	Pergunta que ajuda a identificar sua utilização
Prioridade	Qual tarefa deve receber a CPU primeiro?
Semáforo	Quando uma tarefa deve ser autorizada a continuar sua execução?
Mutex	Qual tarefa pode utilizar determinado recurso compartilhado neste momento?

1 Atividade 1 – Quem deve executar primeiro?

Objetivo

Investigar experimentalmente o efeito das prioridades no escalonamento das tarefas.

Considere um equipamento embarcado que possui três funções:

- aquisição de sensores;
- atualização de um display;
- transmissão de informações de diagnóstico.

Crie três tarefas:

```
TaskSensor  
TaskDisplay  
TaskDiagnostico
```

Cada tarefa deverá enviar periodicamente uma mensagem pela UART indicando que está executando.

Experimento A – Prioridades iguais

Configure inicialmente as três tarefas com a mesma prioridade.

Observe:

- a sequência de execução;
- a alternância entre as tarefas;
- o comportamento do escalonador.

Registre a saída observada no terminal serial.

Experimento B – Prioridades diferentes

Configure agora:

```
TaskSensor      -> prioridade alta  
TaskDisplay     -> prioridade normal  
TaskDiagnostico -> prioridade baixa
```

Execute novamente o sistema.

Experimento C – Bloqueio voluntário

Introduza chamadas a:

```
1 osDelay(...);
```

nas diferentes tarefas.

Teste diferentes valores de atraso e observe novamente o escalonamento.

Questão para análise

Aumentar a prioridade de uma tarefa significa necessariamente que ela executará mais vezes? Explique utilizando os estados *Running*, *Ready* e *Blocked*.

2 Atividade 2 – Atendimento de uma situação crítica

Objetivo

Compreender como as prioridades podem ser utilizadas para diferenciar tarefas críticas e não críticas.

Considere uma máquina industrial contendo:

- uma tarefa responsável pelo processo;
- uma tarefa responsável pela interface;
- uma tarefa responsável pelo tratamento de uma situação crítica.

Implemente:

```
TaskProcesso  
TaskInterface  
TaskEmergencia
```

Configure inicialmente todas as tarefas com a mesma prioridade.

Posteriormente, execute três experimentos configurando a tarefa `TaskEmergencia` como:

```
1 osPriorityLow  
2 osPriorityNormal  
3 osPriorityHigh
```

Observe como a mudança de prioridade interfere no momento em que a tarefa obtém acesso ao processador.

Desafio

Modifique uma das tarefas de menor importância para que ela utilize intensivamente a CPU. Investigue se é possível prejudicar ou impedir a execução adequada de outras tarefas.

Questão para análise

Uma tarefa possuir prioridade elevada é suficiente para garantir que ela sempre apresentará um pequeno tempo de resposta? Justifique.

3 Atividade 3 – Só processe quando houver dados

Objetivo

Utilizar um semáforo binário para sincronizar a produção e o processamento de dados.

Considere um sensor que produz periodicamente uma nova medição.
Implemente duas tarefas:

TaskSensor

TaskProcessamento

Inicialmente, implemente uma solução baseada em *polling*:

```

1 while(1)
2 {
3     if(novoDado)
4     {
5         processar();
6     }
7 }
```

Observe o comportamento da tarefa.

Em seguida, substitua o mecanismo por um semáforo.

A tarefa que representa o sensor deverá liberar o semáforo:

```

1 osSemaphoreRelease(sensorSemHandle);
```

A tarefa responsável pelo processamento deverá aguardar:

```

1 osSemaphoreAcquire(sensorSemHandle, osWaitForever);
```

A arquitetura resultante deverá representar:

TaskSensor

|

v

Semaforo

|

v

TaskProcessamento

Questão para análise

Compare as soluções com *polling* e semáforo. Qual delas utiliza melhor os recursos do sistema operacional? Explique.

4 Atividade 4 – Recursos limitados

Objetivo

Compreender a utilização de um semáforo contador para representar um conjunto limitado de recursos.

Considere um estacionamento automatizado com apenas três vagas.
Cada veículo deverá ser representado por uma tarefa:

Carro1

Carro2

Carro3

Carro4

Carro5

Crie um semáforo contador representando as três vagas disponíveis.
Antes de entrar no estacionamento, cada tarefa deverá executar:

```
1 osSemaphoreAcquire(vagasHandle, osWaitForever);
```

Ao sair:

```
1 osSemaphoreRelease(vagasHandle);
```

Utilize mensagens pela UART para indicar:

- tentativa de entrada;
- entrada autorizada;
- permanência no estacionamento;
- saída;
- liberação da vaga.

Experimento

Modifique a quantidade de recursos disponíveis:

$3 \rightarrow 2 \rightarrow 1$

Observe o comportamento das tarefas.

Questão para análise

Quando um semáforo contador possui apenas uma unidade disponível, qual é o comportamento observado? Ele se torna equivalente a um mutex? Discuta as diferenças conceituais.

5 Atividade 5 – Compartilhamento da UART

Objetivo

Investigar problemas decorrentes do acesso concorrente a um recurso compartilhado.

Considere duas tarefas:

TaskSensor

TaskControle

Ambas precisam utilizar a mesma UART.

Inicialmente, faça as duas tarefas enviarem mensagens relativamente longas sem qualquer mecanismo de proteção.

Exemplo:

```
1 for(int i = 0; i < 50; i++)
2 {
3     printf("SENSOR...");
4 }
```

e:

```
1 for(int i = 0; i < 50; i++)
2 {
3     printf("CONTROLE...");
4 }
```

Observe o resultado no terminal serial.

Posteriormente, crie um mutex para controlar o acesso à UART:

```
1 osMutexAcquire(uartMutexHandle, osWaitForever);
2
3 /* Regiao critica */
4 /* Utilizacao da UART */
5
6 osMutexRelease(uartMutexHandle);
```

Compare os resultados.

Questão para análise

Por que um mutex é conceitualmente mais adequado para proteger a UART do que um semáforo?

6 Atividade 6 – O contador que apresenta o valor errado

Objetivo

Identificar experimentalmente uma condição de corrida e utilizar mutex para proteção de uma variável compartilhada.

Declare:

```
1 uint32_t contadorGlobal = 0;
```

Crie duas tarefas que executem:

```
1 for(int i = 0; i < 100000; i++)
2 {
3     contadorGlobal++;
4 }
```

Crie uma terceira tarefa responsável por apresentar o valor final do contador pela UART. O resultado teoricamente esperado será:

$$100000 + 100000 = 200000$$

Execute o experimento várias vezes e registre os valores encontrados.

Proteção com mutex

Proteja a operação utilizando:

```
1 osMutexAcquire(contadorMutexHandle, osWaitForever);
2
3 contadorGlobal++;
4
5 osMutexRelease(contadorMutexHandle);
```

Execute novamente o experimento.

Questão para análise

Explique por que a operação

`contadorGlobal++`

não deve ser considerada necessariamente atômica.

Utilize em sua explicação a sequência:

LER → MODIFICAR → ESCREVER

7 Atividade 7 – Se estiver ocupado, faça outra coisa

Objetivo

Investigar diferentes estratégias de espera durante a tentativa de aquisição de um recurso compartilhado.

Considere uma tarefa de controle que precisa utilizar ocasionalmente um recurso compartilhado.

Entretanto, caso o recurso esteja ocupado, a tarefa não pode ficar bloqueada indefinidamente.

Inicialmente utilize:

```
1 osMutexAcquire(recursoMutexHandle, osWaitForever);
```

Observe o comportamento.

Posteriormente implemente:

```
1 if(osMutexAcquire(recursoMutexHandle, 0) == osOK)
2 {
3     /* Recurso disponivel */
4
5     /* Executa regioao critica */
6
7     osMutexRelease(recursoMutexHandle);
8 }
9 else
10 {
11     /* Recurso ocupado */
12
13     /* Continua realizando outras atividades */
14 }
```

Experimente também valores intermediários de *timeout*.
Compare:

`osWaitForever`

`timeout limitado`

`timeout = 0`

Questão para análise

Em uma aplicação de controle em tempo real, por que pode ser inadequado bloquear uma tarefa indefinidamente esperando por um recurso que não é essencial para sua operação principal?

8 Atividade 8 – Mini Sistema Industrial

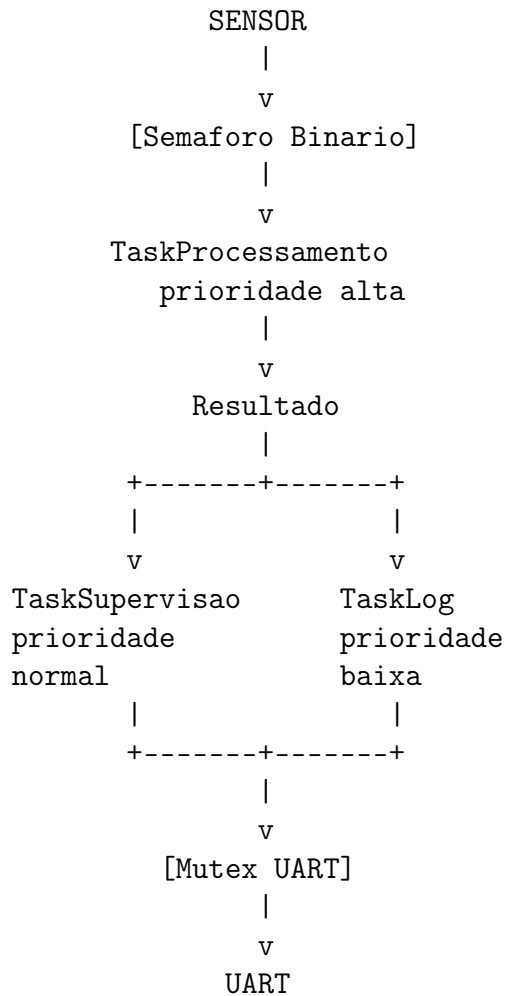
Objetivo

Integrar prioridades, semáforos e mutex em uma pequena aplicação de tempo real.

Considere uma esteira industrial responsável pelo transporte e processamento de peças. O sistema deverá possuir:

- sensor de presença de peça;
- processamento da peça;
- supervisão;
- registro de eventos;
- tratamento de alarmes;
- comunicação UART.

Uma possível arquitetura conceitual é:



O sistema deverá apresentar pelo menos as seguintes tarefas:

TaskSensor
TaskProcessamento
TaskSupervisao
TaskLog
TaskAlarme

Requisitos

O grupo deverá definir:

1. as prioridades das tarefas;
2. quais tarefas deverão utilizar semáforos;
3. quais recursos deverão ser protegidos por mutex;
4. quais tarefas podem permanecer bloqueadas;
5. quais tarefas não podem aguardar indefinidamente;
6. quais informações serão apresentadas pela UART.

Desafio

Introduza propositalmente pelo menos dois problemas de concorrência ou escalonamento no sistema. Demonstre experimentalmente os problemas e posteriormente apresente uma solução utilizando os mecanismos estudados.

Análise final

Preencha a tabela indicando o mecanismo que considera mais adequado para cada situação.

Problema	Prioridade	Semáforo	Mutex
Definir qual tarefa deve executar primeiro	☐	☐	☐
Esperar pela ocorrência de um evento	☐	☐	☐
Proteger uma variável compartilhada	☐	☐	☐
Controlar acesso exclusivo à UART	☐	☐	☐
Representar três recursos disponíveis	☐	☐	☐
Atender rapidamente uma tarefa crítica	☐	☐	☐
Sincronizar sensor e processamento	☐	☐	☐

Questão para análise

Com suas próprias palavras, explique a diferença entre:

1. prioridade;
2. semáforo;
3. mutex.

Para cada mecanismo, apresente um exemplo de aplicação em um sistema embarcado real.

Entrega

Para cada atividade realizada, o grupo deverá apresentar:

1. **Diagrama da aplicação**, mostrando tarefas e recursos;
2. **Código-fonte**, desenvolvido utilizando CMSIS-RTOS v2;
3. **Evidências experimentais**, incluindo capturas do terminal serial;
4. **Descrição do comportamento observado**;

5. **Análise dos resultados**, relacionando o comportamento observado aos conceitos estudados.

Na Atividade 8, o grupo deverá apresentar adicionalmente uma tabela contendo:

Requisito	Mecanismo	Justificativa
-----------	-----------	---------------

Síntese

Ao finalizar as atividades, procure responder à seguinte questão:

Diante de um problema em uma aplicação FreeRTOS, como decidir se a solução exige alterar a prioridade de uma tarefa, utilizar um semáforo ou proteger um recurso utilizando mutex?